

User Study – Manual Python Workflow (Method 1 Only)

Instructions: Fill Section A once. For Section B, answer *per use-case*. Duplicate Section B if you completed multiple use-cases.

Participant ID: use case2-user13 Your assignment: ☒ Method 1 (Manual Python)
Use case(s) you worked on: ☐ U1 River Flow ☒ U2 Smart-Home Energy ☐ U3 Solar Energy

Section A – Background (answer once)

A1. Educational level (*highest completed*)

☐ High School ☐ Vocational/Technical ☐ Bachelor's ☒ Master's ☐ Doctorate ☐ Other:

A2. Primary role (*pick the closest*)

☐ IoT/CPS Developer ☐ Data Scientist / ML Engineer ☐ Embedded SW Engineer ☐ Educator / Researcher ☒ Software Architect / Full-stack ☐ System Integrator ☐ Other:

A3. Self-rated expertise (*tick one per row*)

	Beginner	Intermediate	Advanced	Expert
Python programming	☐	☐	☒	☒
Time-series / ML	☐	☐	☒	☐
Domain knowledge (your use-case)	☐	☒	☐	☐

A4. Prior experience (*used ML frameworks/libraries before?*) ☒ Yes ☐ No

Section B – Task-Specific (duplicate this page *per use-case*)

This B-section is for: ☐ U1 River Flow ☒ U2 Smart-Home Energy ☐ U3 Solar Energy

B1. Time & Effort

Record your own times. Estimate if needed. “Extra learning time” = time spent reading docs/tutorials.

B1. Time spent (minutes)

	Method 1 (Manual Python)
Writing/testing code (end-to-end)	<u>200 minutes</u>
Extra learning time	<u>100 minutes</u>

B2. Lines of code (LoC) (*exclude blanks/comments*)

Manual Python LoC: ~120

B2. Task Completion & Ease of Use

Completion: None = could not finish; Partial = finished some parts; Complete = all parts.

B3. Were you able to finish the use-case with Manual Python?

Method 1 (Manual Python): ☐ None ☐ Partial ☒ Complete

B4. Ease by workflow step (Manual Python) (1=Very easy, 5=Very difficult; tick one per cell)

Step	1	2	3	4	5
Whole workflow	☐	☐	☒	☐	☐
Data preprocessing	☐	☒	☐	☐	☐
Feature engineering (lags/rolling)	☒	☐	☐	☐	☐
Model training	☐	☐	☐	☒	☐
Model evaluation/plots	☐	☐	☒	☐	☐

B1. Which steps did you complete? (check all that apply)

- ☒ Preprocessing
- ☒ Model configuration
- ☒ Training
- ☒ Evaluation
- ☒ Plotting/Diagnostics

B2. How difficult was each step? (1=Very easy, 5=Very difficult; tick one per row)

Step	1	2	3	4	5
Preprocessing	☐	☒	☐	☐	☐
Model configuration	☐	☐	☐	☒	☐
Training	☐	☐	☐	☒	☐
Evaluation	☐	☐	☒	☐	☐
Plotting/Diagnostics	☐	☐	☒	☐	☐

B3. Notes (issues, decisions, or gaps to revisit):

Resampled 1-minute data to 15-minute intervals and simplified SARIMA/SARIMAX to $(1,1,0) \times (1,1,0,96)$ to reduce execution time; skipped daily repeat baseline due to initial data size assumption. Revisit 1-minute cadence, daily repeat, and complex model orders with full dataset for better accuracy.

B5. Time spent per step (record for this use-case; minutes or start/end)

Step	Start (hh:mm)	End (hh:mm)	Minutes
Preprocessing	_____	_____	20 minutes
Model configuration	_____	_____	30 minutes
Training	_____	_____	40 minutes
Evaluation (metrics/baseline)	_____	_____	20 minutes
Plotting / Diagnostics	_____	_____	20 minutes
Debugging (errors, crashes, outputs)	_____	_____	30 minutes
Extra learning / searching / docs	_____	_____	90 minutes
Total			250 minutes

Notes: “Learning” includes reading docs, web searches, tutorials, setup notes, and guides. “Debugging” covers error fixing, crashes, failed runs, and investigating unexpected results.

B6. Comments (bugs, friction points, ideas):

Bugs: Encountered `NameError: rmse_sarima` due to undefined variable and `ValueError: x and y must have same first dimension` from mismatched prediction dimensions, resolved through debugging.

Friction Points: Long execution times with 1-minute data and seasonal period 1440; resampled to 15-minute for feasibility. Daily repeat baseline omitted due to initial data size assumption.

Ideas: Use parallel processing (e.g., `joblib`) to speed up SARIMA/SARIMAX. Test 1-minute cadence with full dataset and add daily repeat baseline for complete comparison.

Train Holt–Winters 5min Test 2min

Train SARIMA 10min Test 3min
Train /SARIMAX 15min Test 5min

Section E – IoT Use Case Template Reuse (Part 2)

Instructions: This section evaluates your experience reusing the provided IoT pipeline template (e.g., River Flow Forecasting) and adapting it for your own use case. Please answer honestly.

E1. Template Understanding

How clear was the IoT pipeline template (overview, scripts, execution order)?

☐ 1 Very unclear ☐ 2 Unclear ☒ 3 Neutral ☐ 4 Clear ☐ 5 Very clear

E2. Adaptation Effort

How easy was it to adapt the template to your chosen use case?

☐ 1 Very difficult ☐ 2 Difficult ☒ 3 Neutral ☐ 4 Easy ☐ 5 Very easy

E3. Reuse of Offline Training Code

How straightforward was it to reuse the provided offline training code with your dataset?

☐ 1 Not at all ☐ 2 A little ☒ 3 Neutral ☐ 4 Mostly ☐ 5 Very straightforward

E4. Time & Effort (minutes) Understanding the template: _____
Adapting scripts (sensors, gateway, analytics): 30min
Running end-to-end pipeline: 120min
10min

E5. Problems & Debugging

Did you face errors while reusing the template? ☒ Yes ☐ No

UserWarnings, briefly described analytical issues to mismatch in feature names during scaler transform (input as numpy array without column names, while fitted with DataFrame).

No alerts triggered initially because simulated sensor data (Temperature/Humidity) led to low predictions (<6 kW threshold); needed to adjust means in sensors.py.

How difficult were they to fix?

☐ 1 Very easy ☐ 2 Easy ☒ 3 Neutral ☐ 4 Hard ☐ 5 Very hard

E6. Usefulness

How useful was the template for accelerating your implementation?

☐ 1 Not useful ☐ 2 Slightly useful ☐ 3 Neutral ☒ 4 Useful ☐ 5 Very useful

E7. Suggested Improvements

What changes would make the IoT template easier to reuse for future use cases?

It seems like a better template should be considered for its implementation. It's explained very vaguely.

Scale anchors (for consistency):

- Clarity / Ease: 1 = Very unclear/difficult, 2 = Unclear/difficult, 3 = Neutral, 4 = Clear/easy, 5 = Very clear/easy.
- Debugging difficulty: 1 = Very easy, 5 = Very hard.
- Usefulness: 1 = Not useful, 5 = Very useful.

User Study – ML2++ (Textual DSL)

Instructions: Fill Section A once. For Section B, answer for *each* use case you worked on. If you did multiple use cases, **duplicate Section B** per use case.

Participant ID: user4 Use case(s): ☐ U1 River Flow ☒ U2 Smart-Home Energy
☐ U3 Solar Energy

Section A – Background (answer once)

This section helps us understand your experience level so we can analyze results fairly.

A1. Educational level (highest completed)

Tell us the highest level of education you have finished.

☐ High School ☐ Vocational/Technical ☐ Bachelor's ☒ Master's ☐ Doctorate ☐ Other: _____

A2. Primary role (pick the closest)

Choose the option that best matches your current job or main activity. If none fits, select “Other”.

☐ IoT/CPS Developer ☐ MDE Practitioner ☐ Data Scientist / ML Engineer ☐ Embedded SW Engineer ☐ Educator / Researcher ☒ Software Architect / Full-stack ☐ System Integrator ☐ Early Adopter / Innovator ☐ Other: _____

A3. Self-rated expertise (tick one per row)

For each area, select your skill level. “IoT/CPS” = connected devices/systems. “MDE” = using models to guide or generate code. “ML/Data Science” = working with data and ML.

	Beginner	Intermediate	Advanced	Expert
IoT/CPS development	☐	☐	☐	☒
Model-Driven Engineering (MDE)	☐	☐	☒	☐
Machine Learning / Data Science	☐	☒	☐	☐

A4. Prior experience (used DSLs/modeling tools/ML frameworks before?)

Have you used similar tools (e.g., modeling editors, DSLs, ML libraries/frameworks)?

☒ Yes ☐ No

Section B – Task-Specific (duplicate this page *per use case*)

This B-section is for: ☐ U1 River Flow ☒ U2 Smart-Home Energy ☐ U3 Solar Energy

If you did more than one use case, copy this whole Section B for each one and fill it separately.

B1. Time & Effort

Please record your times honestly. If unsure, give your best estimate. “Extra learning time” = time reading docs/tutorials to figure things out.

B1. Time spent (minutes)

Minutes you spent defining/editing the DSL, generating code, reviewing outputs, and any extra learning time.

ML2++ (Textual DSL)	
Defining DSL + generate + review	<u>200min</u>
Extra learning time	<u>60min</u>

B2. Statements written (DSL) (exclude blanks/comments)

Approximate number of DSL lines/statements you actually typed or edited.

ML2++ (Textual DSL)	
ML2++ DSL statements	<u>100min</u>

B2. Task Completion & Ease of Use

Tell us if you finished the task and how easy/hard each step felt. Then write how many minutes you spent per step.

B3. Were you able to finish the use case?

“None” = could not complete; “Partial” = finished some parts; “Complete” = finished everything.

☐ None ☐ Partial ☒ Complete

B4. Ease by workflow step (with per-step time)

Rate difficulty (1=very easy, 5=very difficult) and write minutes spent. “Model configuration” = choosing algorithm and hyperparameters. “Model evaluation” = checking results with plots/metrics.

Step	1	2	3	4	5	Time
Whole workflow (end-to-end)	☐	☒	☒	☐	☐	<u>45min</u>
Data preprocessing	☐	☒	☒	☐	☐	<u>8min</u>
Model configuration (algorithm, hyperparameters)	☐	☒	☒	☐	☐	<u>10min</u>
Model training	☐	☒	☐	☐	☐	<u>12min</u>
Model evaluation (metrics/plots/inspection)	☐	☐	☐	☒	☐	<u>20min</u>

Tip: use a simple stopwatch and pause for breaks. If unsure, estimate.

B5. Ease of ML2++ elements (1=Very easy → 5=Very difficult)

Rate how easy it was to work with each part of the language/editor.

Element	1	2	3	4
Data types & objects	☐	☒	☐	☐
Things & behaviours (statecharts)	☐	☐	☒	☐
Ports & properties	☒	☐	☐	☐
Model configuration options (algorithm/hyperparameters)	☐	☒	☐	☐
Configuration (instances/connectors)	☐	☐	☒	☐

B3. Learning & Usability

These questions focus on how quickly you learned the tool and how natural it felt.

B6. Learning curve (1=Very easy → 5=Very difficult)

How much effort did it take to learn enough to finish this use case?

☐1 ☒2 ☐3 ☐4 ☐5

B7. Intuitiveness (1=Very intuitive → 5=Not intuitive)

Did the tool feel natural/obvious to use, or did it feel confusing?

☐1 ☐2 ☒3 ☐4 ☐5

B8. Hardest part to learn (briefly)

Name the single most confusing or time-consuming thing to learn.

B4. Debugging & Error Handling

Tell us about problems you faced and how hard they were to fix.

B9. Error frequency (0=None, 1=Few, 2=Moderate, 3=Many)

How many problems blocked you (errors, failed runs, missing packages, wrong paths, etc.)?

☐0 ☒1 ☐2 ☐3

B10. Clarity of error messages (1=Not clear → 5=Very clear)

When something went wrong, did the messages help you understand the fix?

☐1 ☐2 ☒3 ☐4 ☐5

B11. Debugging effort (1=Easy → 5=Hard)

Overall, how hard was it to fix your issues?

☐1 ☐2 ☒3 ☐4 ☐5

B12. Debugging time (minutes) (estimate)

About how many minutes did you spend fixing problems (total for this use case)?

25min

B5. Code Quality, Maintainability, Flexibility

These questions refer to the generated code you viewed and how easy it would be to change things later.

B13. Generated code quality (1=Poor → 5=Excellent)

How good is the generated code (readable, well-structured, correct)?

☐1 ☐2 ☒3 ☐4 ☐5

B14. Maintainability (1=Very difficult → 5=Very easy)

If you had to update/change your solution later (e.g., new dataset, new horizon), how easy would it be?

☐1 ☐2 ☒3 ☐4 ☐5

B15. Flexibility / Customisation (1=None → 5=Very flexible)

How easily can you change models, hyperparameters, thresholds, or add steps without workarounds?

☐1 ☒2 ☐3 ☐4 ☐5

Section C – Overall Assessment (ML2++ only)

Your overall opinion and which features helped you most.

C1. Overall satisfaction (1=Very dissatisfied → 5=Very satisfied)

Your overall feeling about using ML2++ for this use case.

☐1 ☐2 ☒3 ☐4 ☐5

C2. Would you use ML2++ for your next similar project?

If you had a similar project tomorrow, would you choose this approach? Tell us briefly why.

☐ Yes ☒ No ☐ Undecided Why? Hardest part to learn (briefly)
Name the single most confusing or time-consuming thing to learn.

C3. Feature effectiveness (1=Ineffective → 5=Very effective)

How useful were these features for you?

Feature	1	2	3	4	5
Time-series forecasting models	☐	☐	☒	☐	☐
Automatic preprocessing & plots	☐	☒	☐	☐	☐
Training configuration options	☐	☐	☒	☐	☐
Multi-day forecasting accuracy	☐	☒	☐	☐	☐

C4. Favourite feature: Training configuration options, as they allowed fine-grained hyperparameter tweaks without deep-diving into boilerplate code.

What was the single most helpful feature?

- C5. Least favourite feature:** Automatic plots, since it frequently failed to render prediction plots correctly due to model output mismatches, blocking evaluation.
Which feature felt least helpful or got in your way?
- C6. Missing features you would like:** Built-in validation for model outputs before plotting, and support for ensemble methods to improve forecasting robustness.
What do you wish the tool had (e.g., a plot, algorithm, wizard, or shortcut)?
-

Section D – Open Feedback

Tell us anything else that would help us improve.

D1. Barriers to adoption (*what would stop you from using ML2++ in production?*)

Examples: setup, missing plots, docs, integration, performance, governance.

Inconsistent plotting reliability in model evaluation, plus limited scalability for IoT datasets without better integration hooks.

D2. One improvement you would prioritize

If we could fix/add only one thing, what would help you the most?

Enhance model evaluation with robust, error-resilient plotting that auto-detects and suggests fixes for output format issues.

Scale anchors used throughout for consistency:

- Ease / Learning: 1=Very easy, 2=Easy, 3=Moderate, 4=Difficult, 5=Very difficult.
- Intuitiveness: 1=Very intuitive, 2=Intuitive, 3=Neutral, 4=Somewhat confusing, 5=Not intuitive.
- Error clarity: 1=Not clear, 2=Somewhat clear, 3=Clear, 4=Very clear, 5=Extremely clear.
- Code quality: 1=Poor, 2=Fair, 3=Good, 4=Very good, 5=Excellent.
- Maintainability: 1=Very difficult, 2=Difficult, 3=Moderate, 4=Easy, 5=Very easy.
- Flexibility: 1=None, 2=Limited, 3=Some, 4=Flexible, 5=Very flexible.
- Feature effectiveness: 1=Ineffective, 2=Low, 3=Moderate, 4=Effective, 5=Very effective.